Reducing Interference in MoE Fine-Tuning with Dynamic LoRA

Network and Data Analysis Group

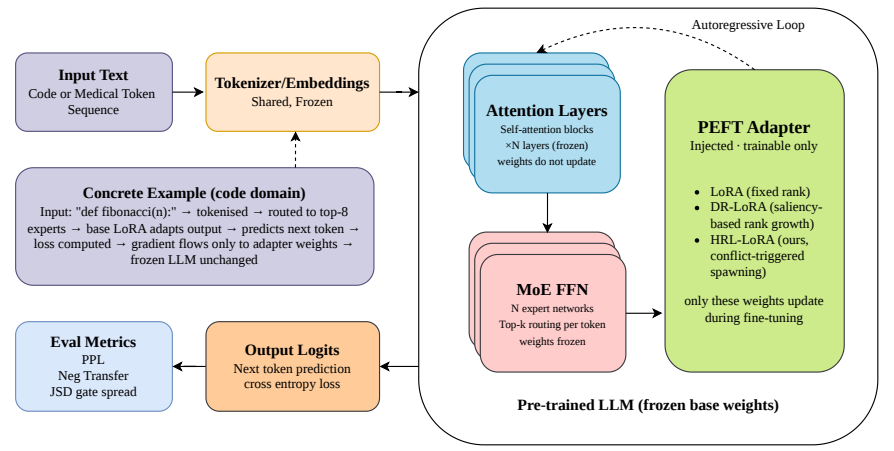
Mehreen Hossain Chowdhury	**210041219**
Ahmed Shafin Ruhan	**210041116**
Nowshin Mahjabin	**210041128**

Supervisor
Abu Raihan Mostofa Kamal
Professor
Dept. of CSE, IUT

Co-Supervisor
Md. Azam Hossain
Professor
Dept. of CSE, IUT

Table of Contents

Background: General PEFT Pipeline for MoE



Background: Key Terms

Mixture of Experts (MoE)

Replaces each FFN with N parallel expert networks. A router selects top-k experts per token. Sparse activation keeps compute constant despite large total parameters.

Parameter-Efficient Fine-Tuning (PEFT)

Fine-tunes only a small set of new parameters while the base model stays frozen. Matches full fine-tuning quality at a fraction of the cost.

LoRA

Injects a trainable low-rank bypass $\Delta W = BA$, $r \ll d$. Only B and A update; base weight W is frozen and merged at inference with zero overhead.

Interference

When learning multiple domains together, updates from one domain disrupt the other, leading to worse performance than training them separately.

Perplexity (PPL) — Primary Metric

Measure of how confused a model is when predicting the next token. PPL = $e^{\mathcal{L}}$, cross-entropy on held-out code. Lower is better.

Negative Transfer

When training on two domains, degrades one:

$$\text{NegT} = \text{PPL}_{\text{code}}(\text{mixed}) - \text{PPL}_{\text{code}}(\text{solo})$$

A positive value means forgetting.

Capacity Fragmentation

A shared rank-r subspace cannot converge toward two orthogonal gradient directions simultaneously. This is the core failure mode we address.

Motivation: Why Standard LoRA Fails in MoE

The Central Problem

When fine-tuning on code + medical data simultaneously, the more medical data we add, the worse the model gets at code performance.

Mix	PPL	Cost
0% medical	430	—
20% medical	748	+317
50% medical	1,209	**+779**

NegTransfer = $\text{PPL}_{\text{code}}(\text{mixed}) - \text{PPL}_{\text{code}}(\text{solo})$

All numbers from our Phi-MoE runs.
Lower PPL = better.

What is happening?

- The MoE router **already separates domains** (Jaccard ≈ 0.056) — routing is not the problem
- Inside each expert, code and medical gradients are **near-orthogonal** (cosine ≈ −0.002)
- A single shared rank-r subspace **cannot converge** toward two orthogonal directions simultaneously - this is **capacity fragmentation**

Goal: Can conflict-triggered, hierarchical sub-adapter spawning inside MoE experts resolve this? And if so, what is the mechanism?

Literature Review: Taxonomy

Papers Relevant to Our Work (Grouped by Theme)

Theme	Papers
MoE Foundations	Sparsely-Gated MoE *(Shazeer et al., 2017)* · Switch Transformer *(Fedus et al., 2022)*
Fixed-Rank PEFT	**LoRA** *(Hu et al., ICLR 2022)* · QLoRA *(Dettmers et al., NeurIPS 2023)* · AdapterFusion *(Pfeiffer et al., EACL 2021)*
Dynamic / Adaptive Rank	**AdaLoRA** *(Zhang et al., ICLR 2023)* [SVD importance] · **DR-LoRA** *(Liu et al., arXiv 2024)* [EMA saliency growth]
Hierarchical / MoE-Specific	**HiLoRA** *(2026)* [static tiers] · MixLoRA *(Zhu et al., 2024)* · Pushing MoE to Limit *(Zadouri et al., 2023)*
Multi-Domain / Continual	MixLoRA-DSI *(EMNLP 2025)* → dynamic expansion for continual learning
Direct Problem Support	**PERFT** *(2026)* → empirically argues MoE requires a *mixture* of modules
Base Models & Datasets	Phi-tiny-MoE-instruct *(Microsoft)* · OLMoE *(AllenAI 2024)* · HumanEval *(Chen et al., 2021)* · PubMedQA *(Jin et al., 2019)*

Literature Review: LoRA (2022)

Hu et al., Microsoft Research, ICLR 2022

What It Does

Freezes W and injects a trainable bypass:

$$\Delta W = BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times d},\ r \ll d$$

At inference, ΔW merges into W at zero latency. Evaluated on RoBERTa, DeBERTa, GPT-2, and GPT-3.

Contributions

- Matches full fine-tuning with <0.1% trainable parameters
- Zero inference overhead
- Rank r gives interpretable capacity control
- Universal PEFT baseline for LLMs

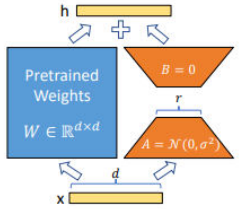


Figure 1: Our reparametrization. We only train A and B.

Limitations Relevant to Our Work

- **Single shared** subspace — all domains compete for same r dimensions
- Cannot separate orthogonal gradient directions
- Rank fixed at initialisation; no response to emerging conflict

Literature Review: AdaLoRA (2023)

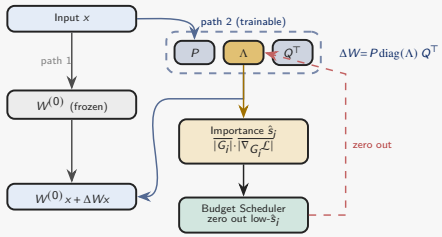
Zhang et al., ICLR 2023

What It Does

Parameterises updates as $\Delta W = P \cdot \text{diag}(\Lambda) \cdot Q^\top$. Tracks singular value importance and **adaptively prunes** low-importance components while reallocating parameter budget across layers.

Contributions

- SVD-based importance scoring for principled rank allocation
- Eliminates manual per-layer rank search
- Improved over fixed-rank LoRA on GLUE and summarisation
- Precursor to saliency-based rank methods (e.g. DR-LoRA)



Drawbacks Relevant to Our Work

- Adaptive allocation still within **a single shared subspace** per weight
- Pruning singular values cannot separate orthogonal domain gradients

Literature Review: DR-LoRA (2026)

Liu et al., arXiv 2026

Method

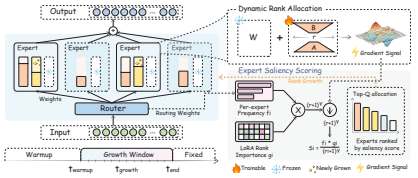
Saliency-driven rank growth. Uses routing EMA + Fisher EMA to identify high-traffic experts. Greedy quota expands rank at salient positions.

Contributions

- Dynamic rank (no manual tuning)
- Capacity follows usage
- Outperforms fixed-rank LoRA
- Strong baseline

Limitation

- Expands rank within the **same shared subspace**, so orthogonal gradients just get more room to compete, not separate paths
- Saliency-based growth fires on importance alone, no awareness of **domain divergence**
- **Result:** NegTransfer +1311.93 at 50% conflict which is 2.3× worse than HRL-LoRA (+574.79)



Dynamically Ranked LoRA

Insight: Capacity ≠ solution

Literature Review: HiLoRA (2026)

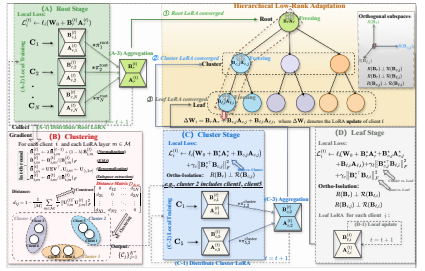
Peng et al., CVPR 2026

What It Does

Hierarchical adapter tiers with coarse-to-fine **static allocation**. Higher-level adapters capture broad task patterns; lower-level adapters specialise to sub-tasks. Structure fixed at init based on task heuristics not, runtime signals.

Contributions

- First hierarchical adapter structure for LLMs
- Coarse-to-fine organisation improves over flat LoRA on multi-task
- Closest analogue to our base + sub-adapter design



HiLoRA: static hierarchical tiers

Limitations and why HiLoRA ≠ HRLoRA

- **Static allocation** → fixed at init, cannot adapt mid-training
- Our sub-adapters are **spawned dynamically** on conflict signals
- Static design fails under evolving task interference

Literature Review: Overview

Prior Work at a Glance

Method	Core Idea	Relevance	Key Limitation
Sparsely-Gated MoE Shazeer et al., 2017	Top-k routing; sparse activation	Establishes the MoE architecture we adapt	Pre-training only; within-expert adaptation unstudied
LoRA Hu et al., 2022	$\Delta W{=}BA$, frozen base, zero overhead	Our base PEFT method; single-adapter baseline	**Single shared subspace - all domains compete**
AdaLoRA Zhang et al., 2023	SVD importance; adaptive rank budget	Lineage precursor to DR-LoRA	**Still one shared subspace; sharpens conflict**
DR-LoRA Liu et al., 2024	Saliency-based rank growth via EMA	Strongest dynamic-rank baseline; directly compared	**Grows shared subspace; amplifies incoherence**
HiLoRA 2026	Hierarchical tiers; static allocation	Structurally similar hierarchy to our design	**Static** - cannot respond to detected conflict
PERFT 2026	MoE demands a *mixture* of modules	**Direct support for our problem statement**	No conflict-triggered spawning mechanism
MixLoRA-DSI EMNLP 2025	Dynamic LoRA expansion for continual learning	Dynamic expansion concept; closest in spirit	Continual learning only, not multi-domain conflict

Research Gaps and Opportunities

What the Literature Collectively Provides

- MoE routing separates domains at expert level. *Within-expert* gradient dynamics remain unstudied
- LoRA, AdaLoRA, DR-LoRA all grow or allocate rank but within a shared subspace, amplifying conflict at each step
- PERFT (2026): MoE empirically *requires* a mixture of modules, not one shared adapter, which is a direct support for our problem statement
- HiLoRA (2026), MixLoRA-DSI (2025): improve flexibility via hierarchical/dynamic LoRA, but only at task/input level, not at intra-expert conflict resolution during training.

Gap 1: Within-Expert Gradient Dynamics

No prior work measures gradient-space interference *inside* individual MoE experts during multi-domain fine-tuning. The router's separation is documented, but per-expert gradient dynamics are not.

Gap 2: Conflict-Responsive Spawning

No method triggers independent sub-adapters based on *joint* saturation **and** domain divergence. Not saliency alone, not static allocation, not a task-level heuristic.

Our approach: Verify empirically (diagnostics) → Resolve structurally (HRLoRA).

Diagnostics: Routing Overlap & Gradient Analysis

Diagnostic 1: Routing Overlap (Jaccard)

Does the router already separate domains?

Domain Pair	Jaccard	Thresh.
Python vs Medical	0.056	0.35
Math vs Fiction	0.069	0.35

High-overlap layers: L26 (0.55), L14 (0.50), L9 (0.50)

Finding

Routing-level conflict is **not** the cause. ≈1 shared expert per layer, 8–12 fully separated. The fix must operate **inside the expert**. Per-layer scores also identify intervention targets: Layers 9, 14, 26.

Diagnostic 2: Gradient Cosine Similarity

Are gradients destructive or capacity-competing?

Metric	Value
Mean cosine similarity	−0.002
Std deviation	±0.003
Fraction negative	75% (15/20)

Finding

Near-perfectly **orthogonal** which signals **capacity fragmentation**, not cancellation. Each step partially satisfies one domain at the expense of the other.

Why This Invalidates DR-LoRA

Orthogonal gradients ⇒ more rank just gives domains more room to compete. Confirmed by our results: DR-LoRA **+1311.93** NegTransfer at 50% conflict.

Research Questions

RQ1: Mechanism

Does within-expert gradient fragmentation cause negative transfer in MoE multi-domain fine-tuning?

▹ Establishes whether the failure mode is a property of the adapter subspace, not the router, motivating intervention inside the expert.

RQ3: Operative Mechanism

Is architectural independence the operative mechanism, not capacity increase?

▹ Distinguishes between the two competing explanations for any observed improvement, with DR-LoRA serving as the key control condition.

RQ2: Method Superiority

Can conflict-triggered spawning outperform both fixed-rank and dynamic-rank adapters?

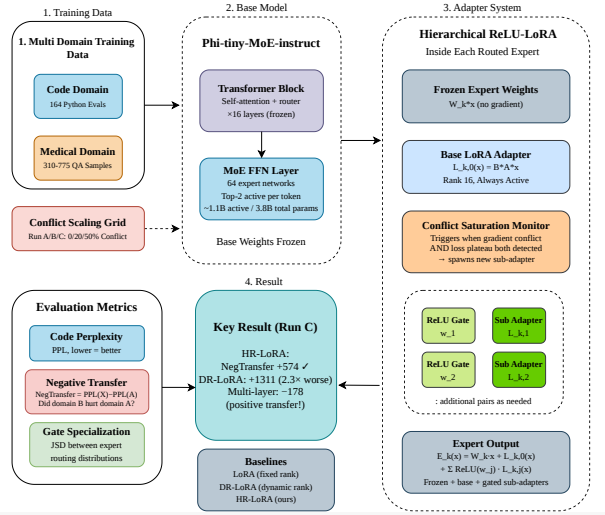
▹ Establishes whether structural independence (new sub-adapters) is more effective than capacity expansion (more rank) when domain gradients are orthogonal.

RQ4: Multi-Layer Targeting

Does Jaccard-guided multi-layer targeting generalise the benefit?

▹ Establishes whether intervening at the highest-conflict layers (identified by Jaccard) produces additive gains, and whether spawning self-localises correctly.

Proposed Method: Architecture



The Entanglement Problem

- Single shared adapter per expert cannot handle orthogonal gradient directions
- Result: negative transfer **scaling with conflict**

Proposed Solution: HR-LoRA

- **Base LoRA** - always active
- **Sub-adapters** - spawned on demand
- **ReLU gates** - activate per domain

Key insight: Structural independence, not just more capacity.

Proposed Method: Forward Pass

Forward Pass for Expert k

$$E_k(x) = W_k x + \underbrace{L_{k,0}(x)}_{\text{base LoRA}} + \sum_j \underbrace{\mathrm{ReLU}(w_{k,j}^\top x)}_{\text{gate}} \cdot \underbrace{L_{k,j}(x)}_{\text{sub-adapter}}$$

Base LoRA $L_{k,0}$

Always active for every token. Captures general domain-agnostic adaptation. Rank 16, SVD-initialised from residual gradient.

ReLU Gate $w_{k,j}$

A learned vector per sub-adapter. $\mathrm{ReLU}(w_{k,j}^\top x)$ produces a scalar that **activates or silences** the sub-adapter per token. Gates specialise by domain (confirmed by JSD ≈0.35–0.43).

Sub-Adapter $L_{k,j}$

Spawned on demand when conflict detected. B=**0** init guarantees zero loss impact at spawn. Learns an **independent** subspace for the conflicting domain.

Why Not Just More Rank?

- DR-LoRA expands rank within the **same shared subspace**. This way, orthogonal gradients just get more room to compete
- HRL-LoRA creates **structurally independent** paths (confirmed by results: DR-LoRA +1311.93 vs. HRL-LoRA +574.79 NegTransfer at 50% conflict)

Proposed Method: Spawn Trigger

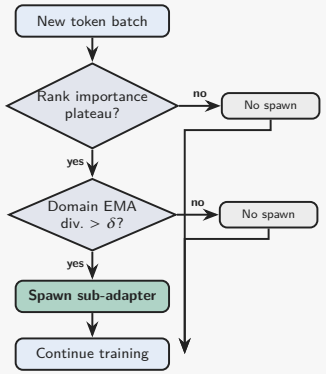
When Does Spawning Occur?

A new sub-adapter is spawned only when **both** conditions hold simultaneously:

1. **Rank importance plateau:** moving-avg importance of singular components stalled ($\Delta\text{imp} < \tau = 10^{-4}$, window 15 steps)
2. **Domain EMA divergence** $> \delta$**:** EMA losses for code and medical batches diverged beyond δ=1.0 nats

Trigger Parameters

δ (divergence threshold)	1.0 nats
τ (plateau threshold)	10^{-4}
EMA window	15 steps
Max sub-adapters per expert	10
Min interval between spawns	100 steps



≈**1 spawn per 100 steps.** Bivariate condition keeps spawning rare and targeted.

Mathematical Validation

Three properties must hold to ensure spawning is well-posed and results are interpretable.

1. Zero-Loss Spawn

Claim: spawning causes no change to the current loss.
Why: new sub-adapter's B initialised to $\mathbf{0}$, so $L_{k,j}(x) = \mathbf{0}$ at spawn time.
Verified: $|\Delta\text{Loss}| = 0.000000$ across all spawn events in our runs.

2. Dead-Zone Escape

Claim: despite B=0, the adapter begins learning within steps.
Why: ReLU gate $w_{k,j}$ is separate from B and receives non-zero gradients immediately.
Verified: gate gradient norm $= 6.989 \times 10^{-3}$ at step+2.

3. Router Invariance

Claim: adding sub-adapters leaves global routing unchanged.
Why: router logits $r(x)$ computed before expert computation.
sub-adapters modify $E_k(x)$ but not $r(x)$.
Verified analytically. Any PPL change is attributable to adaptation alone.

Together: (1) spawning cannot hurt performance at the moment it occurs, (2) the adapter will begin learning immediately despite zero-init, and (3) any measured improvement in PPL or NegTransfer is causally attributable to the sub-adapter's domain separation, not a routing side-effect.

Datasets

HumanEval: Code Domain

Chen et al., 2021 (OpenAI)

- 164 Python programming problems
- Includes function signatures, docstrings, and unit tests
- Used as code-domain training source
- **Why:** clean, self-contained Python. maximally distinct from biomedical prose

PubMedQA: Medical Domain

Jin et al., EMNLP 2019

- Biomedical QA from PubMed abstracts
- 310 examples for Run B (20% conflict)
- 775 examples for Run C (50% conflict)
- **Why:** lexically and syntactically distant from Python, which produces strong, reliable conflict signal

Conflict-Scaling Design: Run A: 0% medical (code-only ceiling) Run B: 20% medical (moderate conflict) Run C: 50% medical (severe conflict)
The three-run grid isolates the effect of increasing domain conflict on code-side perplexity for each method.

Experimental Setup

Model: Phi-tiny-MoE-instruct

`microsoft/Phi-tiny-moe-instruct` (Microsoft). Sparse MoE architecture. All results in this deck are from runs on this model. Extension to OLMoE-1B-7B is planned as future work.

Methods Compared

- **LoRA:** fixed rank, no growth
- **DR-LoRA:** saliency rank growth; 6 events at steps 337–1272; router unfreezes at step 150
- **HRLoRA (ours):** bivariate trigger (δ=1.0, cap=10)

Conflict-Scaling Design

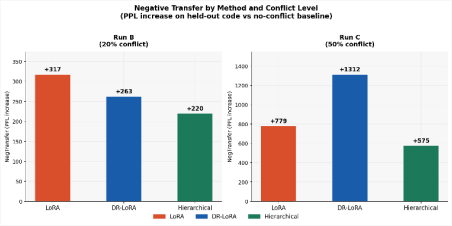
- **Run A:** 0% medical - code-only baseline
- **Run B:** 20% medical - moderate conflict
- **Run C:** 50% medical - severe conflict

Evaluation Metrics

- **PPL_code** - Primary Metric; lower is better
- **NegTransfer** = PPL(X) − PPL(A)
- **JSD** of gate activations

Key Results

Method	Run	Conf.	Code PPL	NegTransfer
LoRA	A	0%	430.60	—
LoRA	B	20%	747.85	+317.25
LoRA	C	50%	1209.38	+778.78
DR-LoRA	A	0%	461.10	—
DR-LoRA	B	20%	723.61	+262.51
DR-LoRA	C	50%	1773.03	+1311.93
HRLoRA	A	0%	464.92	—
HRLoRA	B	20%	**684.52**	**+219.60**
HRLoRA	C	50%	**1039.72**	**+574.79**

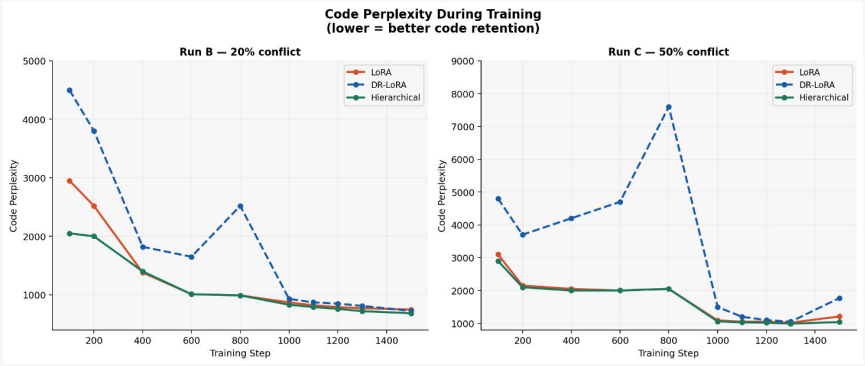


Run A equivalence: all methods converge to 430–465 PPL without conflict. Differences in B/C reflect conflict handling only.

Expansions: HRLoRA spawns 10 sub-adapters in B and C. LoRA: 0. DR-LoRA: 1 rank expansion event.

DR-LoRA collapse at Run C: +1311.93 NegTransfer which is 2.3× worse than HRLoRA. Saliency expansion into a shared subspace amplifies gradient incoherence under severe conflict.

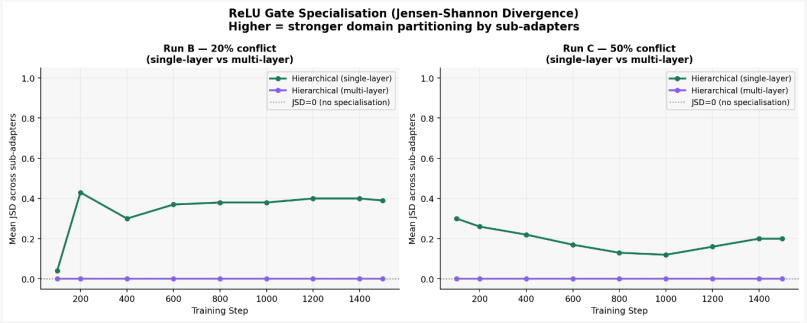
Results: Code Perplexity Trajectories



Run B (20% conflict): HRL-LoRA converges to the lowest final PPL (≈600). DR-LoRA shows erratic spikes at its rank growth events, indicating instability around step 900.

Run C (50% conflict): DR-LoRA spikes catastrophically to >7500 PPL at step 800 when the router unfreezes under severe conflict. HR-LoRA and LoRA track together before HR-LoRA pulls ahead.

Results: ReLU Gate Specialisation (JSD)



Run B: gates develop strong specialisation (JSD ≈0.43 by step 200) and stabilise around 0.35–0.40. Sub-adapters are partitioning domains, not merely accumulating capacity.

Run C: specialisation is lower (≈0.20–0.30), consistent with more severe conflict requiring more steps to stabilise. Multi-layer extension (purple) shows no spawning in non-target layers which confirms **self-localisation**, not a null result.

Ablation: Eager Spawn (Trigger Timing)

What We Tested

Does lowering $\delta : 1.0 \rightarrow 0.5$ recover performance under severe conflict (Run C)?

Config	First Spawn	PPL	NegT.
Standard (δ=1.0, cap=10)	Step 48	1039.72	+574.79
Eager (δ=0.5, cap=15)	Step 47	973.05	+508.13

Both configs reached their sub-adapter caps.

Finding

- Halving δ moved first spawn by **one step only**
- Improvement is marginal
- Trigger sensitivity is **not** the binding constraint

Corrected Interpretation

- Binding constraint is the **cap** (10), not δ
- All sub-adapters exhausted early under severe conflict
- No architectural headroom remains for rest of training

Implication

Cap should scale with conflict intensity:

$$\text{cap} = \lfloor 5 + 10 \cdot \tanh(|\Delta_{\text{EMA}}|) \rceil$$

- More sub-adapters under severe conflict
- Fewer under mild conflict

Ablation: Multi-Layer Extension

What We Tested

- Extend HRL-LoRA to top-3 Jaccard layers: **L26** (0.5513), **L14** (0.5025), **L9** (0.5000)

Config	PPL	NegT.	Spawns
Single-layer (L0)	684.52	+219.60	10 (L0)
Multi-layer (9,14,26)	**286.22**	**−178.70**	10 (L14)

Key Findings

- **58% PPL reduction** vs. single-layer
- NegTransfer turns **negative** (−178.70) multi-domain training *improved* code performance
- All spawning concentrated in Layer 14, trigger self-localises to highest-conflict layer
- Confirms Jaccard-guided targeting is the correct intervention strategy

Self-localisation: given three candidate layers, spawning activated only in the one most diagnostic of conflict. The diagnostic correctly predicts where adaptation pressure concentrates.

Summary of Contributions

Contribution 1: Diagnostic Framework

Jaccard + gradient cosine analysis established that MoE domain entanglement is a **gradient-space capacity fragmentation** problem, not routing-level. A methodological contribution independent of the proposed method.

Contribution 2: HRLoRA

Three validated properties (zero-loss spawn, dead-zone escape, router invariance). Lowest NegTransfer at both conflict levels:

- Moderate: **+219.60** vs. LoRA +317.25, DR-LoRA +262.51
- Severe: **+574.79** vs. LoRA +778.78, DR-LoRA +1311.93

Contribution 3: DR-LoRA Failure Analysis

DR-LoRA **fails under severe conflict** (+1311.93). Confirms that architectural independence, not capacity expansion, is the operative mechanism. Shared-subspace expansion compounds incoherence.

Contribution 4: Multi-Layer Extension

58% PPL reduction over single-layer via Jaccard-targeted layers. Spawning self-localised to Layer 14, confirming entanglement is concentrated and the diagnostic correctly identifies intervention sites.

Limitations & Future Work

Limitations

- **Metric scope:** code perplexity only, pass@k not yet complete for HRL-LoRA
- **Cap sensitivity:** fixed cap of 10 is the binding constraint under severe conflict; dynamic formula proposed but unvalidated
- **Domain scope:** single domain pair only, other combinations unstudied
- **Seed confound:** multi-layer result uses a different random seed; matched replication required before formal publication

Future Work

- **Functional evaluation:** complete pass@1 on HumanEval to confirm lower PPL translates to better code generation
- **Dynamic cap:** validate cap = $\lfloor 5 + 10 \cdot \tanh(|\Delta_{\text{EMA}}|) \rceil$ to scale sub-adapters with conflict intensity
- **OLMoE extension:** run full conflict-scaling grid on OLMoE-1B-7B to validate generalisation
- **Domain pairs:** replicate on code + creative writing, multilingual, and three-domain mixtures

Conclusion

Summary

HRLoRA Spawning evaluated on Phi-tiny-MoE-instruct under a conflict-scaling grid. Diagnostics established domain entanglement as a **gradient-space capacity fragmentation** problem, not routing-level.

Key Results

- Lowest NegTransfer at both levels: **+219.60** (20%), **+574.79** (50%)
- DR-LoRA collapses at 50%: +1311.93, shared-subspace expansion counterproductive
- Multi-layer: **58% PPL reduction**, NegTransfer turns **negative** (−178.70)
- ReLU gates sustain genuine domain specialisation (JSD ≈0.35–0.43)

Core Finding

- **Architectural independence** is the operative mechanism
- Expanding a shared subspace is insufficient, or actively harmful
- The separation must be **structural**

Thank You

References (1/2)

MoE Foundations

[1] Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer.* ICLR 2017.

[2] Fedus, Zoph, Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* JMLR 2022.

[3] Lepikhin et al. *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding.* ICLR 2021.

PEFT - Fixed Rank

[4] Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022.

[5] Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs.* NeurIPS 2023.

[6] Pfeiffer et al. *AdapterFusion: Non-Destructive Task Composition for Transfer Learning.* EACL 2021.

[7] Houlsby et al. *Parameter-Efficient Transfer Learning for NLP.* ICML 2019.

PEFT - Dynamic / Adaptive Rank

[8] Zhang et al. *AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning.* ICLR 2023.

MoE-Specific Adaptation

[10] Zhu et al. *MixLoRA: Enhancing LLM Fine-Tuning with LoRA-based MoE.* arXiv 2024.

[11] Zadouri et al. *Pushing Mixture of Experts to the Limit.* arXiv 2023.

[12] *HiLoRA: Hierarchical Low-Rank Adaptation.* 2026.

[13] *PERFT: Parameter-Efficient Routing for MoE Fine-Tuning.* 2026.

[14] *MixLoRA-DSI: Dynamic LoRA Expert Expansion.* EMNLP 2025.

PEFT Surveys

[15] Ding et al. *Parameter-Efficient Fine-Tuning of Large-Scale Pre-Trained Language Models.* Nature Machine Intelligence, 2023.

[16] He et al. *Towards a Unified View of Parameter-Efficient Transfer Learning.* ICLR 2022.

[17] Han et al. *Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey.* arXiv 2024.

References (2/2)

Base Models

[18] Microsoft. *Phi-tiny-MoE-instruct* (`microsoft/Phi-tiny-moe-instruct`). HuggingFace, 2024.

[19] Muennighoff et al. *OLMoE: Open Mixture-of-Experts Language Models.* AllenAI, arXiv 2024.

[20] Jiang et al. *Mixtral of Experts.* arXiv 2024.

[21] Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone.* arXiv 2024.

Datasets

[22] Chen et al. *Evaluating Large Language Models Trained on Code* (HumanEval). arXiv 2021.

[23] Jin et al. *PubMedQA: A Dataset for Biomedical Research Question Answering.* EMNLP 2019.

Multi-Domain & Continual Learning

[24] Wang et al. *Orthogonal Fine-tuning via Butterfly Factorization.* ICLR 2024.

[25] Chronopoulou et al. *AdapterSoup: Weight Averaging to Improve Generalization of Pretrained Language Models.* EACL 2023.

[26] Yu et al. *Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch.* ICML 2024.

Gradient Interference & Negative Transfer

[27] Yu et al. *Gradient Surgery for Multi-Task Learning.* NeurIPS 2020.

[28] Fifty et al. *Efficiently Identifying Task Groupings for Multi-Task Learning.* NeurIPS 2021.

[29] Winata et al. *Overcoming Catastrophic Forgetting in Massively Multilingual Continual Learning.* ACL 2023.